Wavel: A Fast and Efficient Compilation System for Wafer-Scale Accelerators

Yeqi Huang
University of Edinburgh

Congjie He
University of Edinburgh

Haocheng Xiao
University of Edinburgh

Yanwei Ye
University of Edinburgh

Yi-Chieh Wang
University of Edinburgh

Boyao Song
University of Edinburgh

Yangshen Deng
University of Edinburgh

Ziming Miao
Microsoft Research

Lingxiao Ma
Microsoft Research

Fan Yang
Microsoft Research

Luo Mai
University of Edinburgh

Abstract

Wafer-scale accelerators offer a new scaling point for AI infrastructure, but they also create a new compilation regime: communication cost varies sharply with location, and the space of possible placements and execution schedules is enormous. Existing GPU, distributed, and vendor compilation systems largely retain a sharding-oriented view and therefore fail to fully leverage these emerging accelerators, leaving the dominant physical scheduling decisions unresolved.

We make a key observation: at wafer scale, the central complexity of compilation shifts from finding a logical shard plan to constructing an *executable schedule* that fixes placement, layout alignment, and execution ordering. Based on this observation, we present Wavel, a fast and efficient compilation system for wafer-scale accelerators. Wavel introduces MeshIR to make executable schedules explicit, constructs them through two legal locality-preserving primitives, `CUT` and `TUNE`, and uses physical cost evaluation with structured pruning to keep search practical. On WSE-3 hardware, Wavel delivers 1.31×–1.35× higher mean per-setting throughput than the expert-optimized WaferLLM and up to 1.85× speedup over the Full-Chip baseline on dense models; calibrated cost-model results predict a 2.97× speedup on MoE workloads over the same baseline.

Keywords: AI accelerators, compilers, spatial architectures

ACM Reference Format:
Yeqi Huang, Congjie He, Haocheng Xiao, Yanwei Ye, Yi-Chieh Wang, Boyao Song, Yangshen Deng, Ziming Miao, Lingxiao Ma, Fan Yang, and Luo Mai. 2026. Wavel: A Fast and Efficient Compilation System for Wafer-Scale Accelerators. In *Symposium on Operating Systems Principles (SOSP '26), September 29–October 2, 2026, Prague, Czech Republic.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3830418.3843884



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '26, Prague, Czech Republic
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2585-2/26/09
https://doi.org/10.1145/3830418.3843884

1 Introduction

Wafer-scale accelerators are emerging as an important platform for AI infrastructure, as reflected by their recent adoption by OpenAI [20], AWS [3], Meta [18], and Mistral [5]. Their impact extends beyond LLM inference: applications such as high-performance computing and vector search could also benefit substantially, potentially achieving 100–200× speedups from tens of PB/s of on-wafer memory bandwidth and tens of dense PFLOPS, provided that the software stack can unlock this performance.

Unlike today's AI accelerators, wafer-scale systems organize up to one million cores, each with private memory and a router, into a large on-chip mesh rather than relying on shared-memory hierarchies and off-chip interconnects. This distributed, mesh-centric design is motivated by scaling efficiency: the interconnect cost of shared-memory systems grows quadratically with core count [2], whereas 2D on-chip meshes scale more favorably and enable much larger accelerators through wafer-scale integration [11, 27].

As these systems become more widely adopted, programmers need a wafer-scale compilation system that can expose their benefits without requiring hardware-specific programming. Developers typically express computations over logical tensors without specifying how tensors should be partitioned, placed, and communicated across the wafer's many cores. Bridging this gap requires a compilation system that can map such high-level programs onto the hardware efficiently and within practical compilation time. This problem is substantially harder at wafer scale: the space of core mappings and communication patterns is enormous, far larger than on today's GPUs, where software has highly limited placement choices and benefits from a shared global memory.

This difference in architecture, together with the much larger mapping space, helps explain why practically useful wafer-scale compilation systems remain scarce today. Existing compilation systems largely inherit system architectures from PyTorch and TensorFlow/XLA, including wafer-scale vendor stacks such as Cerebras Torch [4] and smaller-mesh stacks such as Tenstorrent Torch [28], as well as GPU-oriented ones such as Roller [37], Welder [25], and Ladder [31]. These systems either do not support wafer-scale accelerators, or require hours to weeks to produce schedules for LLM workloads while still leaving the hardware heavily under-utilized. As a result, high performance on today's wafer-scale systems often still depends on manual expert mapping, as evidenced by WaferLLM [7], which in turn limits support for emerging model architectures and other wafer-scale applications.

We observe that a central reason prior designs fall short is that they adopt a *sharding-oriented view* of compilation. This view is reasonable for GPUs: the compiler mainly explores logical tensor sharding, while runtime mapping onto SMs has limited impact because computation ultimately executes over a shared global memory system. At wafer scale, this balance changes. Because communication cost varies sharply with location, producer–consumer layouts may mismatch, and logically independent kernels may still conflict on physical regions, a logical partition plan is no longer sufficient. The complexity of wafer-scale compilation shifts from finding a logical sharding to constructing an *executable schedule*. The right search object is therefore not merely a shard plan, but an executable schedule that fixes placement, layout alignment, and execution ordering.

A second observation is that many wafer-scale applications already align naturally with this design shift. Deep neural networks, AI-oriented scientific computing, and vector search pipelines all contain structured and localized interactions that can align with the wafer's physical locality. We refer to this property as *logical–physical locality synergy*. This observation sharply constrains the search: schedules that violate workload locality are usually unpromising and need not be explored.

Guided by these observations, we design and implement Wavel, a fast and efficient compilation system for wafer-scale accelerators. Wavel searches directly over executable schedules rather than incomplete shard plans, generates new states only through legal locality-preserving refinements, and uses physical cost evaluation together with structured pruning to make search practical at wafer scale.

We make the following contributions in Wavel:

- **MeshIR: a representation for efficient search over executable schedules.** MeshIR introduces Regions as rectangular blocks of cores, MeshTensors that bind tensors to Regions, MeshKernels that bind operators to execution regions, and MeshSchedules that make placement, layout alignment, and resource conflicts explicit in a single executable schedule object.
- **Practical search of executable schedules for wafer-scale devices.** Wavel shows that executable schedules can be derived through two core primitives for wafer-scale compilation: `CUT` and `TUNE`. Over these executable states, physical cost evaluation, operator-aware local refinement, and decomposition-based pruning make search practical at wafer scale.
- **An effective implementation integrated with current compiler infrastructure.** Wavel integrates with MLIR to reuse mature optimizations before spatial mapping, supports multiple frontends, and lowers executable schedules to vendor runtimes.

We implement Wavel on top of MLIR with support for Cerebras WSE-3 and Tenstorrent Blackhole. Using this implementation, Wavel compiles real workloads automatically, including LLM inference, scientific simulation, and vector search. Our evaluation shows that Wavel outperforms state-of-the-art manually optimized wafer-scale LLM systems such as WaferLLM while automatically producing schedules for large LLM and MoE models (GPT-OSS, Qwen3, and LLaMA3). On WSE-3 hardware, Wavel delivers 1.31×–1.35× higher mean per-setting throughput than WaferLLM and up to 1.85× speedup over the Full-Chip baseline on dense models; calibrated cost-model results predict up to 2.97× over the same baseline on MoE workloads. Beyond LLMs, Wavel enables challenging particle simulation and vector search workloads that prior compilation systems do not support, executing them end to end in the WSE-3 SDK's cycle-level simulator and yielding 15–31× lower per-step time and 6–59× lower query latency, respectively, relative to an idealized iso-compute GPU-cluster reference under the assumptions in §6.1.1.

The design also extends beyond wafer scale: on Tenstorrent Blackhole, Wavel compiles the evaluated Qwen3.5 and LLaMA3.1 models in 9.8–24.2 seconds, compared with 668.7–682.0 seconds for the vendor pipeline. Device-side performance depends on model-specific kernel support: TTNN is faster on LLaMA, while Wavel is up to 6.9× faster on Qwen.

2 Background and Motivation

2.1 Why Wafer-Scale Accelerators?

Wafer-scale accelerators are emerging as an important platform for low-latency and high-throughput AI inference. As illustrated in Figure 1, they differ from conventional GPUs and smaller spatial accelerators in both memory organization and interconnect structure. Instead of relying on shared-memory hierarchies and off-chip communication, wafer-scale systems organize massive numbers of compute tiles into a single on-wafer 2D network-on-chip (NoC). Each tile contains a

compute unit, a small private memory, and a router, and communication proceeds through hop-by-hop message passing across neighboring tiles.

This architecture creates a different performance regime. In shared-memory systems, increasing core counts places growing pressure on the memory hierarchy and global interconnect. In wafer-scale systems, communication remains on chip, but performance becomes highly sensitive to where computation is placed and how data moves across the mesh. Recent advances in advanced packaging and wafer-scale integration have made such devices practical at unprecedented scale. The Cerebras WSE-3, for example, integrates 950,000 compute cores and 44 GB of aggregated on-chip memory in a single mesh-connected device. Smaller mesh accelerators such as Tenstorrent Blackhole share some of these structural properties, but at a much smaller scale.

Wafer-scale architectures are particularly well suited to *large-scale spatial workloads*, especially LLM inference, in which computation is naturally distributed across many interacting partitions rather than concentrated around a shared-memory bottleneck [26]. Other examples include deep neural network inference beyond LLMs, certain scientific simulations, and vector-search pipelines. In these workloads, communication patterns are often structured and localized, making them a strong match for mesh-connected execution substrates.

2.2 Why Wafer-Scale Compilation Is Hard

Wafer-scale compilation is difficult because it combines fine-grained locality, very large device scale, and a rapidly growing schedule space. Table 1 contrasts these three factors across single-GPU, IPU, GPU-cluster, and wafer-scale settings. These difficulties become especially severe under a sharding-oriented view (summarized in Table 2), which focuses on logical partitioning first and leaves much of the physical realization to later stages.

Problem 1: Fine-grained locality dominates schedule quality. On a wafer-scale mesh, communication cost depends directly on spatial distance. Nearby tiles can exchange data cheaply, while communication across the wafer may traverse hundreds of hops and become orders of magnitude slower. Locality therefore affects nearly every major compilation decision, including placement, data movement, and communication routing.

	Single GPU	IPU	GPU Cluster	Wafer Scale
Locality levels	1	2	few (< 10)	many (up to 1000)
#cores/devices	$\sim 10^2$	$\sim 10^3$	$10^2 \sim 10^4$	$\sim 10^6$
Mapping space	limited	moderate	large	enormous

Table 1. Illustrative comparison of mapping complexity across accelerator settings. Wafer-scale systems combine much finer-grained locality with much larger device scale, making both schedule quality and search efficiency substantially harder.

This is qualitatively different from prior settings. GPU systems mainly expose shared-memory and off-chip bandwidth constraints. IPUs expose a much coarser distinction between local and remote memory. GPU clusters have only a small number of communication tiers. In contrast, wafer-scale systems expose fine-grained locality across a very large spatial substrate. Under a sharding-oriented view, this problem is hard to control because a logical partition says little about the actual hop distances and locality violations of its physical realization.

Problem 2: Wafer scale makes a full model mapping combinatorially hard. A wafer-scale compilation system must decide which regions execute which kernels, how tensors move between regions, and how execution is staged to satisfy memory and routing constraints. Even after per-kernel regions are chosen, the system must still construct a global schedule that respects both data dependencies and resource dependencies.

This problem is harder than in prior DNN compilers for GPUs [6, 8, 17, 22, 25, 31, 33, 36, 37], distributed on-chip systems such as T10 [16], and multi-GPU systems such as Alpa [34] and nnScaler [14]. Under a sharding-oriented view, the difficulty is amplified because the logical plan underspecifies exactly those physical interactions—layout mismatch, region overlap, and asynchronous interference—that dominate execution quality at wafer scale.

Problem 3: Compilation time is itself a systems bottleneck. At wafer scale, it is not enough to eventually find a good schedule; the system must do so quickly enough to remain practical. Developers routinely iterate on model structure, operator choices, batch shapes, and serving configurations. If compilation takes too long, the hardware becomes difficult to program regardless of eventual runtime performance.

This issue is also worse under a sharding-oriented view. If the compiler first explores logical plans and only later resolves their physical realization, much of the search effort is spent on candidates whose true execution cost is still unknown. At wafer scale, where the hidden physical schedule space is itself enormous, this separation makes both schedule quality and compilation time harder to control.

3 Design Intuitions

Our key design intuition is that the search object at wafer scale should be an *executable schedule*, not a logical shard plan. Figure 2 illustrates why: even after a logical partition is chosen, the system must still determine where tensors and kernels reside, how producer–consumer layouts are spatially aligned, and whether apparently independent kernels

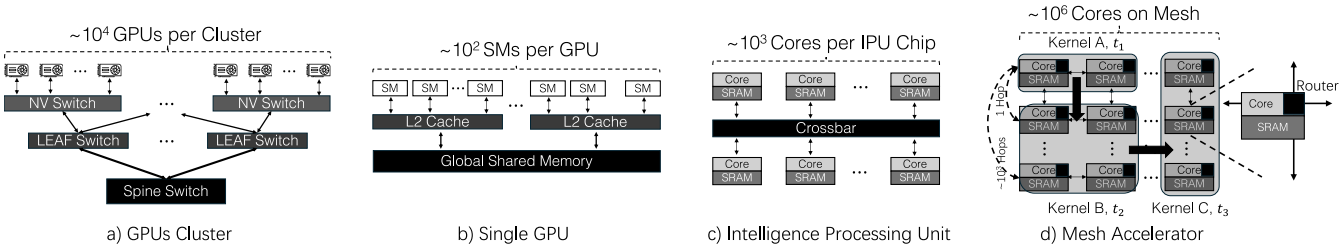


Figure 1. Comparison of memory and interconnect organization across accelerator architectures. GPUs rely on shared-memory hierarchies and centralized interconnects; smaller spatial accelerators distribute memory but remain limited in scale; wafer-scale accelerators expose a large 2D mesh of compute tiles with private memories and hop-by-hop communication. This shift makes spatial locality and placement central to performance and compilation.

Aspect	Sharding-oriented	Wavel
Primary search object	Logical shard / partition plan	Executable region schedule
Meaning of a schedule	Partition is defined; many execution details remain implicit	Placement, spatial alignment, and resource conflicts are explicit
Schedule expansion	Search over shard choices, then rely on later lowering/fix-ups	Apply legal locality-preserving refinements (CUT/TUNE)
Backend responsibility	Resolve missing execution details during lowering	Lower an already executable schedule to target-specific code

Table 2. Wavel shifts the central compilation abstraction from logical shard plans to executable schedules, making placement and alignment explicit.

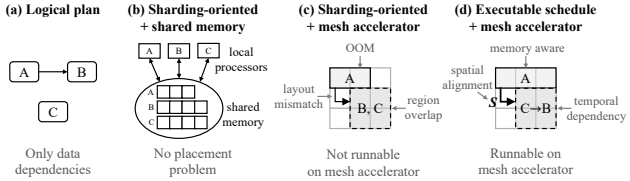


Figure 2. From logical plan to executable schedule. A logical plan (a) works on shared-memory systems (b) but leaves layout mismatches, resource conflicts, and memory violations implicit on a mesh (c); the executable schedule (d) resolves all three.

can actually execute concurrently on overlapping physical regions.

This reframing directly addresses the three problems in §2. It makes locality explicit in the schedule object, restricts search to legal locality-preserving states instead of arbitrary shard plans, and enables more faithful physical cost evaluation over states that are already close to real hardware execution. Table 2 summarizes this shift: Wavel searches over executable schedules, expands them only through legal refinements, and leaves backend lowering to realize an already executable schedule.

Wafer-scale applications already align with this physically grounded design view. Many wafer-scale applications naturally match a compilation view that reasons about physical placement directly. In LLM inference, even within a single attention layer, a small set of dependent matrix multiplications and layout transforms repeatedly exchange intermediate tensors with one another rather than with arbitrary operators across the device. When those communicating operators are mapped to neighboring wafer regions, the dominant data movement remains short-range. We refer to this alignment as *logical–physical locality synergy*.

As Figure 2 illustrates, a sharding-oriented logical plan defines *how work is divided* but cannot execute directly on a mesh because three critical details remain implicit: (1) a tiled region may lack sufficient SRAM, requiring *memory-aware placement* over a larger physical area; (2) producer and consumer data layouts may mismatch, requiring explicit *spatial alignment*; and (3) operators assigned to overlapping regions cannot run concurrently, requiring explicit *temporal ordering*. These gaps only surface once operators are mapped to concrete physical regions.

Wavel avoids this by constructing schedules directly in physical space: every candidate state already encodes legal placement and locality, so the search never wastes effort on plans that would need costly repair.

Open design challenges. These intuitions turn the above problems into three concrete design challenges for Wavel.

Challenge 1: How should the compilation system represent executable schedules? The system needs an abstraction whose valid states already make placement, spatial alignment, and resource conflicts explicit, rather than leaving them implicit in a logical partition plan.

Challenge 2: How should the compilation system generate new feasible candidate schedules? The system needs a small set of feasible locality-preserving refinement rules to expand one executable schedule into another, rather than exploring arbitrary shard plans first and later repairing.

Challenge 3: How can search over executable schedules remain practical? The physical schedule space is vast, and compilation must be fast enough for iterative development. This requires a cost model that can cheaply estimate the runtime of a candidate schedule from its physical properties (region sizes, memory usage, and hop distances), and a way to prune unpromising candidates early by exploiting the structure of the executable schedule space.

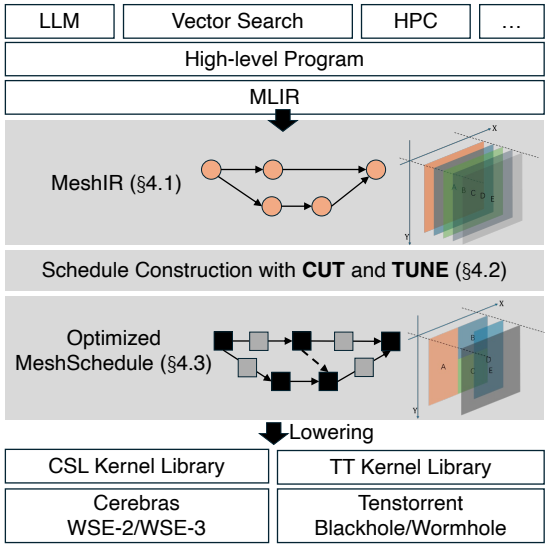
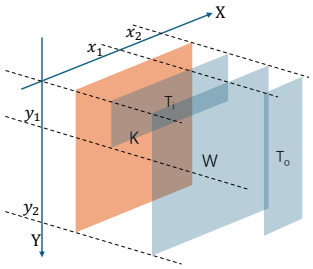


Figure 3. Overview of Wavel. A dataflow graph enters MeshIR, where CUT and TUNE construct executable schedules over explicit physical regions; structural properties of the resulting search space make wafer-scale optimization tractable; a thin backend layer ports the schedule to different mesh targets.

```
class Region{
  Tuple < Int, Int > top_left_coor;
  Tuple < Int, Int > width_height;
}
class MeshTensor {
  Region region;
  Tensor data;
}
class MeshKernel {
  Vector < MeshTensor > input;
  Vector < MeshTensor > output;
  Vector < Region > region_list;
  Compute(func, region_list, input);
}
```

(a) MeshKernel abstraction



(b) MeshKernel K for GEMV

Figure 4. MeshIR building blocks for a mesh kernel. Regions define physical rectangles, MeshTensors bind logical tensors to regions, and MeshKernels bind computation to the regions that execute it.

4 Wavel System Design

Figure 3 gives an overview of Wavel. MeshIR represents executable schedules with explicit placement and alignment (Section 4.1); CUT and TUNE advance one legal schedule to another (Section 4.2); structural properties of the resulting space make search practical (Section 4.3); and a thin backend layer ports the design across mesh targets (Section 4.4).

4.1 Parameterizing Scheduling Space via MeshIR

MeshIR is designed for parameterizing the search space for executable schedules. In this search space, a key element is

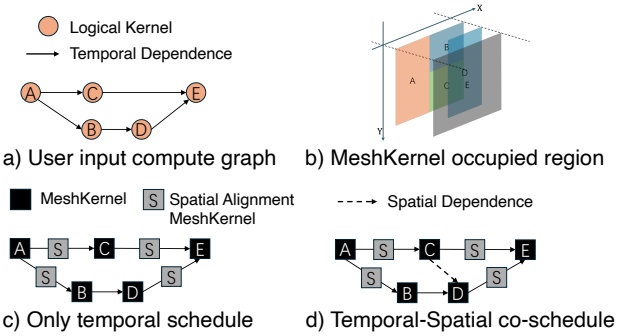


Figure 5. A MeshSchedule makes spatial alignment and temporal ordering explicit.

MeshTensor, which describes how a logical tensor is associated with a physical region, as illustrated in Figure 4. Each MeshTensor carries a Region object that describes locality (i.e., the top left coordinate of the region) and area (i.e., the width and height). Because placement information is part of MeshIR, later schedule construction can reason explicitly about where data already resides and where alignment is required. Formally, a **Region** is a rectangular block $\mathcal{R} = \big((x, y), (w, h)\big) \in \mathbb{N}^2 \times \mathbb{N}^2$, denoting $w \times h$ cores with top-left corner (x, y). A MeshTensor binds a logical tensor to a specific Region, making its physical placement explicit in MeshIR.

MeshKernel binds computation to physical regions. It takes a set of input **MeshTensor**s, applies a compute function (such as GEMV, GEMM, or allreduce), and produces output **MeshTensor**s. Its `region_list`[1] specifies the physical region(s) the kernel occupies during execution: concretely, it is the enclosing region that covers all input, output, and intermediate buffer placements, so that every core involved in the computation falls within the allocated area.

Figure 4 illustrates a MeshKernel K for a GEMV computation. We represent its structure and placements compactly:

```
MeshKernel (K)
⊢ region_list: [R = ((0,0), (x2, y2))]
⊢ input: Ti: [((0,0), (x2, y1))], W: [((0,0), (x2, y2))]
⊢ output: To: [((x_1,0), (x2 − x1, y2))]
⌞ function: GEMV (Ti × W → To)
```

The central search object in Wavel is the **MeshSchedule**, which pairs a compute graph with a concrete spatial arrangement on the mesh:

```
MeshSchedule (S)
⊢ compute_graph: G
⌞ mesh_kernels: [K1, K2, ..., Kn]
```

Here G encodes the computation order among operators (Figure 5(a)), and each K_i already carries its Region $\mathcal{R}_i$ as

[1]In Mixture-of-Experts (MoE) models, certain operators require non-rectangular occupied areas (e.g., T-shaped or L-shaped layouts) to facilitate efficient reduce operations. Because each Region is rectangular, `region_list` represents such shapes as compositions of multiple rectangular sub-regions.

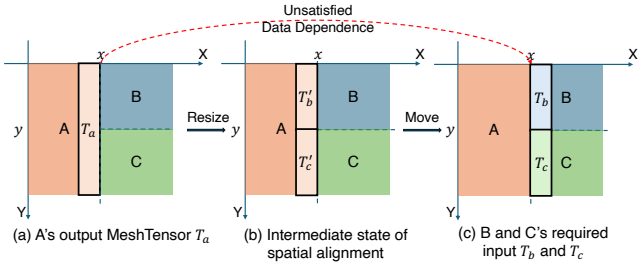


Figure 6. Producer and consumer tensor layouts can mismatch even when the logical dependence is correct. Wavel makes the required spatial alignment explicit rather than leaving it to backend repair.

defined above, determining where each operator executes on the mesh (Figure 5(b)).

MeshKernel spatial alignment. Executability on a wafer-scale mesh requires more than logical dependence. The output region of a producer may not match the input region expected by its consumers. For example, in Figure 5(c), A's output T_a must be delivered to B and C as input, but their regions differ. An explicit spatial-alignment MeshKernel is therefore required at each producer–consumer boundary. Wavel realizes such alignment via Resize followed by Move (Figure 6): first reshaping the tensor layout to match the consumer's expected region size, then transferring the data to the consumer's physical region. As shown in Figure 6, the region of C's input MeshTensor mismatches that of T_a ($\mathcal{R}_{T_c} \neq \mathcal{R}_{T_a}$). To align this, Wavel inserts an alignment MeshKernel $S_{a\to c}$, whose function is Resize and Move.

Additionally, even data-independent kernels may still conflict if their execution regions overlap (Figure 5(d)). Wavel therefore analyzes region occupancy and inserts explicit resource-ordering edges (i.e., temporal ordering constraints) when concurrent execution would be illegal. These edges matter because they place physical interference in the same schedule object that already records logical dataflow and spatial alignment.

Together, Regions, MeshTensors, and MeshKernels define the building blocks; MeshSchedule assembles them into an executable schedule whose feasibility already reflects both communication and resource constraints. This representation also exposes a concrete search interface: once all region descriptors are explicit, an executable MeshSchedule can be parameterized as a structured physical state rather than a logical shard plan.

4.2 Schedule Construction with CUT and TUNE

MeshSchedule defines the *candidate schedule*; the compilation system still needs a transition system that advances one executable schedule to another. Wavel introduces two schedule-construction rules, CUT and TUNE. Each rule is a *legal locality-preserving refinement*: applied to an executable

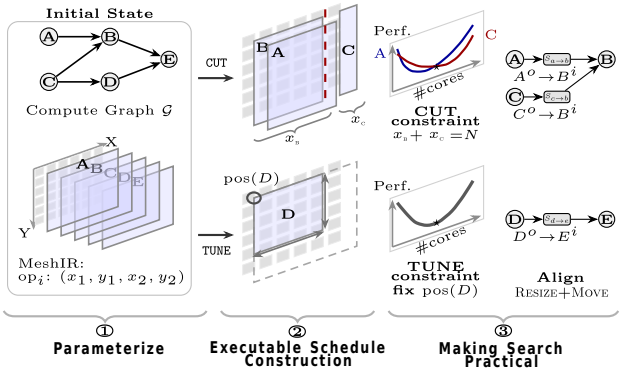


Figure 7. Structured search over executable MeshSchedules. Explicit geometry exposes a searchable state, CUT decomposes topology, and TUNE refines geometry within the resulting subproblems.

schedule, it yields another executable schedule. Together, CUT and TUNE define the feasible successors of any given MeshSchedule.

4.2.1 CUT: topological decomposition.

CUT applies when two operators share the same region. It partitions the region along one axis, assigning one sub-region to each operator. The CUT **constraint** requires that the two resulting sub-regions share a full boundary edge: their union equals the original region and they do not overlap. Figure 7 (❷ top row) illustrates a concrete example: operators B,A are separated from C along the X-axis, with the constraint $x_B + x_C = N$ preserving total width. After CUT, Wavel inserts alignment MeshKernels (e.g., $S_{a\to b}$, $S_{c\to b}$) along the new region boundaries to spatially align producer–consumer layouts. Requiring adjacency after CUT is key to minimizing data movement: because the two sub-regions share a boundary, producer–consumer communication between them stays short-range rather than crossing the mesh.

4.2.2 TUNE: geometric refinement.

TUNE refines the geometry of a single operator while keeping the schedule topology fixed. It selects one corner of the operator's region as an *anchor vertex*, chosen based on the producer in the compute graph to minimize data movement. The TUNE **constraint** requires that the anchor vertex remain fixed while the region's width and height vary within the feasible range imposed by memory and occupancy constraints. Figure 7 (❷ bottom row) shows an example: pos(D) is fixed as the anchor while the remaining boundaries of D's region are tuned. As with CUT, any layout mismatch introduced by the resized region is resolved via alignment MeshKernels (e.g., $S_{d\to e}$). The anchor serves the same purpose as the adjacency constraint in CUT: by pinning the corner closest to the producer, TUNE keeps the data movement path short as the region geometry changes.

Expressiveness of CUT and TUNE. Together, CUT and TUNE define the reachable schedule class from any initial whole-device MeshSchedule. CUT generates hierarchical region decompositions; TUNE adjusts each operator's region within its partition. The combination covers recursively partitioned devices in which every operator has a contiguous, independently sized region. For nonrectangular requirements such as MoE expert routing, `region_list` composes multiple rectangular subregions per MeshKernel. Wavel also supports operator fusion: fused operator groups become compound MeshKernels whose aggregate footprint participates normally in schedule search.

4.3 Making Search Practical

The executable-state representation and the CUT/TUNE transition system together enable three properties that make wafer-scale search tractable: compositional cost evaluation over fully placed states, unimodal cost structure under geometric refinement, and hierarchical decomposition with translation-invariant reuse.

4.3.1 Compositional cost evaluation. Each MeshKernel is bound to a concrete physical region, so the cost of a single operator under given compute resources can be profiled or analytically modeled directly; there is no ambiguity about placement or layout. Combined with MeshSchedule's explicit representation of operator connections (spatial alignment kernels, resource-ordering edges), these per-operator costs compose into a faithful end-to-end estimate for the full compute graph. This is a substantially more reliable evaluation target than an incomplete logical shard plan whose missing execution details must be guessed.

4.3.2 Unimodal cost structure. A key empirical property that accelerates search is the *unimodal* cost structure of many operators with respect to region size. When an operator is assigned too few PEs, computation dominates; when assigned too many, communication overhead dominates. This produces a unimodal curve with a single optimum—an observation consistent with prior work on wafer-scale inference [7] and confirmed in our own measurements. Different operator types exhibit different cost profiles: elementwise and allgather operations are monotonic in region size, while GEMM and GEMV exhibit clear unimodality. Wavel exploits this structure by applying specialized one-dimensional search (e.g., ternary search) when a subproblem's cost curve is unimodal, avoiding exhaustive enumeration of region sizes (Figure 7, ❸).

4.3.3 Decomposition and translation-invariant reuse. CUT simultaneously decomposes the global scheduling problem and shrinks the search space: each subproblem involves fewer operators over a smaller region, making it substantially more tractable than the original (Figure 7, ❸). Wavel further exploits a property of nominally uniform mesh fabrics: *translation invariance*. The best schedule found for a set of operators within a fixed-area region depends only on the region's dimensions, not its absolute position on the mesh. For example, an LLM layer that fits well in a 200×200 region at the top-left corner can be reused identically at any other 200×200 location. Wavel records solved subproblems in a hash table keyed by operator set and region dimensions, enabling direct reuse across spatially distinct but structurally identical subproblems.

4.4 Portability across backends

The system-design boundary in Wavel is simple: design fixes schedule semantics, while backend lowering realizes those semantics on a target runtime. Figure 8 shows where this boundary falls in the compilation stack. MeshIR, MeshSchedule, and the CUT/TUNE transition semantics are preserved across backends because they capture the executable structure of mesh scheduling itself. Backend-specific components remain below this boundary: movement primitives, operator performance modeling, and code generation all depend on the target architecture and runtime interface.

This boundary is what gives the design its portability. Porting Wavel to a new mesh accelerator does not require redefining the search object or the legal refinement rules; it requires providing a lowering path that realizes an already executable schedule and profiling operator performance on the new hardware. Section 5 describes how the current implementation instantiates this boundary on Cerebras and Tenstorrent.

5 Wavel Implementation

We have implemented Wavel as a complete end-to-end compiler stack with roughly 40,000 lines of Python for the frontend, IR infrastructure, and search engine, plus 2,000 lines of C++ to accelerate performance-critical search operations. The system uses Python decorators and JIT tracing to capture computation graphs and annotations transparently, accepting programs from standard tensor frameworks.

Frontend and IR integration. The implementation lowers high-level tensor programs into MLIR before spatial mapping. This lets Wavel reuse standard graph optimizations such as simplification and canonicalization before entering MeshSchedule construction, while keeping mesh-specific schedule semantics in MeshIR rather than in ad hoc backend passes. In practice, MLIR [12] handles the hardware-agnostic graph pipeline, and Wavel takes over once compilation reaches the stage where placement, spatial alignment, and resource conflicts must be made explicit.

Heterogeneous backend code generation. Wavel needs to lower a unified `MeshSchedule` onto hardware platforms whose execution models can differ. This gap matters because schedule decisions depend heavily on how each accelerator

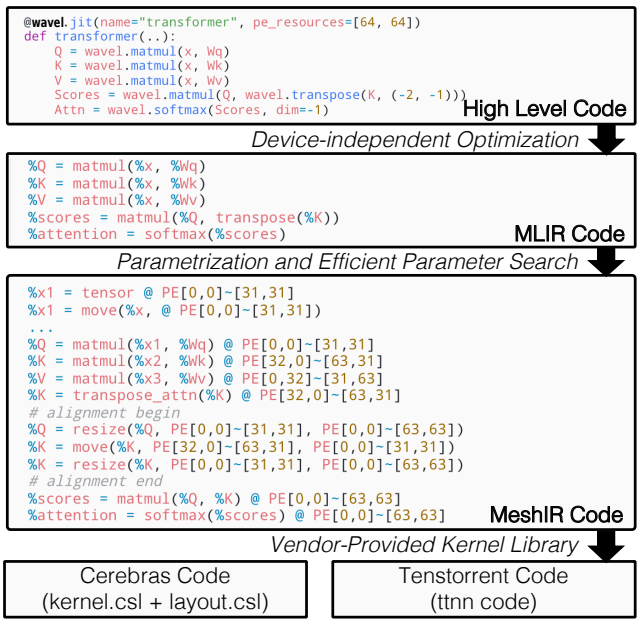


Figure 8. Wavel compilation stack. The frontend constructs typed MeshIR programs; MeshIR and the CUT/TUNE search layer are shared across targets; cost models and data-movement calibration are backend-specific.

handles locality, routing, and kernel/buffer execution. Cerebras exposes a *pure dataflow* architecture where wavelets trigger computation along statically colored routes. Tenstorrent uses a *controlled dataflow* model that requires explicit orchestration of circular buffers, reader/writer kernels, and runtime dispatch arguments. As a result, even simple logical actions compile differently: a "Move" from Region A to B becomes a static routing program on Cerebras, but expands into explicit data movement at the kernel level on Tenstorrent. Wavel addresses this by adopting backend-specific lowering strategies while keeping MeshSchedule and the CUT/TUNE search fixed. For Cerebras, a template-driven backend translates region placement and movement decisions into statically routed CSL programs. For Tenstorrent, a dispatch-oriented backend lowers the same schedule into reader, writer, and compute kernels, circular buffers, and runtime arguments. MeshSchedule and CUT/TUNE therefore form the shared executable contract, while performance models, data-movement calibration, and code generation remain backend-specific.

Key techniques in accelerating search. To navigate the large wafer-scale scheduling space efficiently, we extend A* with domain-specific heuristics. These heuristics leave the schedule space unchanged: every state they reach is a state that CUT and TUNE alone could construct.

First, Wavel directly fixes low-complexity subproblems and otherwise expands a small action set: TUNE, spatial or temporal CUT, and two composite special cases (parallel DUPLICATE and PLACE) that shortcut common CUT/TUNE sequences for replicating an operator and pinning a region. TUNE searches feasible region sizes, while CUT and DUPLICATE expose cross-operator parallelism; candidates that exceed SRAM capacity are discarded, and only dependency-ready groups advance. Second, Wavel memoizes states using a translation-invariant structural key. Thus, two schedules with the same operator grouping and relative geometry are recognized as identical even when rooted at (x_1, y_1) and (x_2, y_2), avoiding repeated exploration of spatially isomorphic states. Bounded A* orders the remaining candidates by predicted cost.

6 Evaluation

We have evaluated Wavel extensively using real commodity hardware. The key findings are:

- **High schedule quality:** On WSE-3 hardware, Wavel achieves 1.31×–1.35× higher mean per-setting throughput than the state-of-the-art wafer-scale inference system WaferLLM, despite finding schedules automatically while WaferLLM's mapping is hand-designed and partitions no finer than a whole layer. Wavel also achieves up to 1.85× speedup over the Full-Chip baseline on dense models; calibrated cost-model results predict up to 2.97× on MoE workloads.
- **Practical structured search:** Wavel combines executable-state construction with structural restrictions that sharply reduce the analytical candidate space, while its end-to-end compilation is up to 70× faster than the evaluated vendor pipeline on Tenstorrent Blackhole.
- **Broad workload coverage:** Wavel generalizes automatically across diverse AI model architectures, including both dense models and Mixture-of-Experts models, as well as other wafer-scale applications such as particle simulation and vector search. Its benefits also extend beyond wafer scale: the same search design improves schedules on smaller mesh accelerators such as Tenstorrent Blackhole.

Hardware. We evaluate Wavel primarily on the **Cerebras WSE-3** using measurements from real hardware, the official Cerebras SDK simulator, and a calibrated performance model. Dense-LLM results, including the WaferLLM comparison, are measured on WSE-3 hardware using SDK 2.10. The SDK simulator measures the one-layer case study and schedule ablation. We use the calibrated model for MoE results. Figure 13 compares model predictions with real-hardware measurements.

To test portability beyond wafer scale, we also evaluate Wavel on three Tenstorrent mesh accelerators: **Grayskull** (120 cores, 120 MB SRAM, 118.4 GB/s DRAM bandwidth), **Wormhole** (72 cores, 108 MB SRAM, 288 GB/s DRAM bandwidth), and **Blackhole** (120 Tensix cores on the die, of which an 11×10 grid is available for mapping, 1.5 MB L1 SRAM per core, 32 GB GDDR6). These platforms feature smaller meshes

and off-chip memory hierarchies, testing whether Wavel remains effective.

6.1 End-to-end Performance on Wafer-Scale Engines

We compare Wavel against two baselines. The first is **WaferLLM**, the state-of-the-art hand-optimized inference system for WSE-3. The second is **Full-Chip**, a strong no-search baseline we implemented in Wavel.

We port WaferLLM's open-source implementation to WSE-3 and rerun it under matched model, precision, batch-size, sequence-length, and runtime settings.

WaferLLM does not partition work inside a transformer layer: every operator runs on the same PE rectangle, serially, so the only mapping freedom is the choice of that rectangle. We reproduce that design decision rather than a schedule search, sweeping 27 candidate placements (18 square and 9 non-square, from 48×48 to 750×994) and reporting the best; the space is coarse enough that exhaustive enumeration is cheap. We run the selected configurations on WSE-3 hardware under the matched settings. Upstream WaferLLM does not support the other dense architectures, so we extend it ourselves, keeping its layout and collective structure and hand-placing every operator, RWKV4's recurrent block included, under the same whole-layer discipline. They are WaferLLM-style mappings of our own construction, not upstream results. We do not extend it to MoE, whose sparse expert activation has no whole-layer analogue.

Full-Chip uses the same optimization passes as Wavel but treats the wafer as a single shared-memory device, assigning each operator to all available cores; this is the natural strategy of a sharding-oriented compiler, so it isolates exactly the benefit of Wavel's schedule search.

We also attempted to include the Cerebras inference engine and its closed-source Torch as two baselines, but their supported model lists are highly limited and do not include the models evaluated here.

Dense LLM Performance. Using the decode and prefill results measured on WSE-3 hardware in Figure 9, we report end-to-end throughput for LLaMA3-8B, LLaMA3-13B,[2] Qwen3.5-27B, Qwen3-VL-8B, and RWKV4-7B. Across the four plotted settings (decode and prefill at 128, 2K, and 4K tokens), Wavel achieves **1.31×–1.35×** higher mean per-setting tokens/s than the best WaferLLM whole-layer placement. We compute this by averaging the per-setting speedup over those four settings for each model, then reporting the range of those per-model means; the interval is therefore an envelope across models, not a confidence interval. Against the Full-Chip baseline, Wavel achieves up to **1.85×** higher throughput on dense models. The **2.97×** maximum occurs on a MoE workload and is reported in Figure 10; it is a calibrated cost-model result rather than a hardware measurement.

[2]LLaMA3-13B refers to the community Llama-3-13B-Instruct checkpoint.

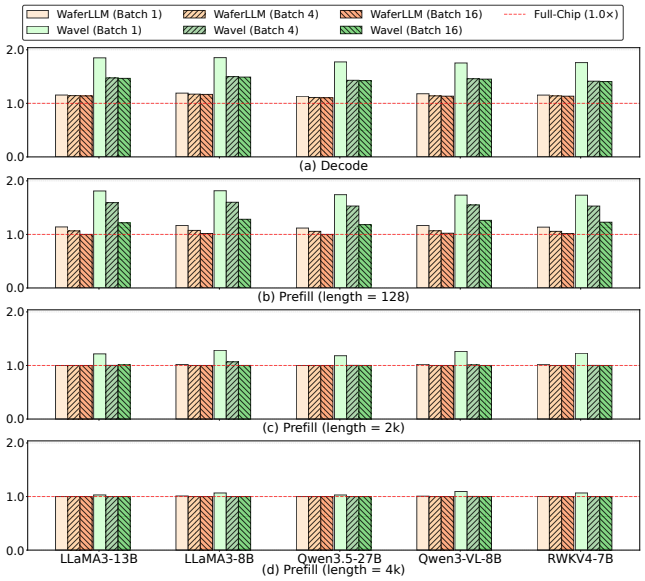


Figure 9. End-to-end dense-LLM inference performance measured on WSE-3 hardware, normalized to the Full-Chip baseline (1.0×).

To understand where these gains come from, we run a controlled one-layer LLaMA3-8B decode case on a 360×360 WSE-3 submesh in the official Cerebras SDK simulator. Consistent with its whole-layer discipline, the WaferLLM baseline executes Q, K, and V serially on the whole region. Starting from the same region, Wavel applies `CUT` to place the three projections on adjacent 120×360 regions and execute them concurrently. Later, Gate and Up execute concurrently on two 180×360 regions. This controlled simulator case exposes the spatial decomposition directly, and it isolates the one degree of freedom WaferLLM's mapping does not expose: partitioning within a layer.

Wavel delivers larger gains in decoding than in prefilling. Decoding is memory-bandwidth-bound, so region placement has a much larger impact. Prefilling is compute-bound, where spreading kernels across all cores is already near-optimal; at large batch sizes, every operator must occupy nearly the full chip to fit tensors on-chip, leaving little room for schedule optimization.

Larger Mixture-of-Experts Models. We further evaluate Wavel on larger MoE models, including Qwen3-30B-A3B and GPT-OSS-20B. Because WaferLLM does not support MoE, we compare only against the Full-Chip baseline in Figure 10. Wavel achieves up to **2.97×** speedup in the decode phase, where sparse expert activation creates severe communication imbalance under the Full-Chip mapping. For these models we claim automatic schedule generation and calibrated cost prediction rather than measured executable code generation: MeshIR represents the forward pass through the experts a token selects, while top-k gating and its dynamic dispatch stay outside the compiled schedule (§8).

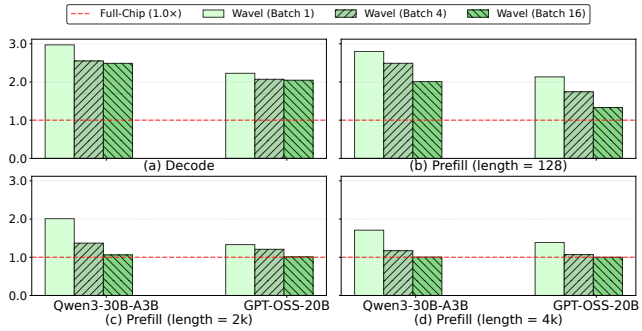


Figure 10. Calibrated cost-model estimates of MoE model performance on WSE-3, normalized to the Full-Chip baseline. Wavel achieves large decode speedups by right-sizing expert regions and parallelizing independent experts.

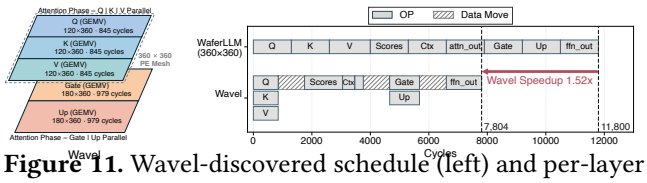


Figure 11. Wavel-discovered schedule (left) and per-layer execution timeline measured in the official Cerebras SDK simulator (right) for LLaMA3-8B decode on WSE-3. Wavel places $Q/K/V$ on three 120×360 regions and Gate/Up on two 180×360 regions; the plotted timeline reports a 1.52× speedup over the uniform 360×360 WaferLLM schedule.

Wavel outperforms the baseline mainly for two reasons. First, unlike the Full-Chip mapping, which assigns every operator to the entire device and often pushes execution into a communication-bound regime, Wavel identifies the right number of cores for each operator. This is particularly impactful for MoE models: because only a subset of experts is activated per token, the effective GEMV and GEMM operations within each expert are relatively small. Using the full chip for these small operators wastes resources and inflates communication overhead, making it especially important to search for the right PE region size. Second, Wavel discovers that independent operators can be spatially separated and executed in parallel rather than sequentially. Although this requires explicit *spatial alignment* kernels to transform data layouts across regions, our evaluation shows that the latency reduction from parallel execution substantially outweighs the overhead of these local data movements.

How Wavel discovers optimizations. Figure 11 illustrates the 360×360 case. `CUT` exposes cross-operator parallelism by partitioning the initial region into three equal Q/K/V bands and two equal Gate/Up bands. `TUNE` then right-sizes the remaining operators within the same mesh. Although each projection receives only part of the device, their overlap shortens the SDK-simulated timeline relative to the uniform WaferLLM schedule. This case makes the trade-off between per-operator cost and cross-operator parallelism explicit. The

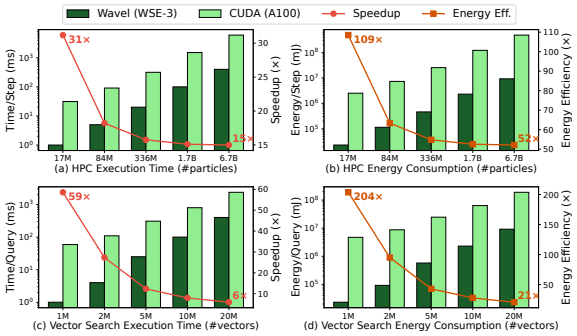


Figure 12. Cycle-level simulated PIC and vector-search performance on WSE-3 relative to an idealized iso-compute 200-A100 reference.

plotted 1.52× speedup is measured in the official Cerebras SDK simulator, not on silicon hardware.

6.1.1 Other Applications: HPC and Vector Search.

Beyond transformers, we study two workloads: a Particle-In-Cell (PIC) simulation with intensive communication, and large-scale vector similarity search. We do not claim that WSE-3 is universally superior to GPU clusters: the platforms differ in architecture, memory hierarchy, and price. Instead, these cases show that MeshIR and `CUT`/`TUNE` can compile non-LLM dataflow workloads with different communication patterns. For both applications, Wavel's frontend compiles reference implementations into operator graphs, applies the scheduling framework automatically, and executes the generated programs end to end in the vendor SDK's cycle-level simulator. We report the resulting simulated WSE-3 performance. For quantitative context, we compare against an iso-compute GPU cluster under optimistic linear-scaling assumptions;[3] readers should treat the ratios as schedule context, not a controlled compiler comparison.

PIC simulation. Figure 12(a)(b) reports simulated WSE-3 PIC performance: 15–31× lower per-step time than the idealized 200-A100 reference and 52–108× higher estimated energy efficiency across 17M–6.7B particles. PIC couples local stencil updates, irregular particle–mesh interactions, and global FFT solves; the generated MeshSchedule represents these phases with explicit regions and alignment operations. The WSE-3 time comes from end-to-end simulator execution, whereas the GPU reference is modeled; thus, the ratios do not establish measured cluster speedup.

[3]We use a *scaled-out* cluster of 200 NVIDIA A100 GPUs under an *iso-compute* assumption. Specifically, we begin with a single-node A100 implementation and extrapolate to 200 GPUs assuming ideal linear scale-out, no additional communication overhead, and perfect overlap. This is intentionally an optimistic upper bound for GPU performance, so the resulting ratios should be interpreted conservatively. Under this assumption, both systems provide comparable peak FLOP capacity (WSE-3: 125 PFLOPS; 200×A100: 124.8 PFLOPS, using A100's sparse FP16 peak), while energy results use realistic power envelopes (WSE-3: 23 kW; 200×A100: 80 kW).

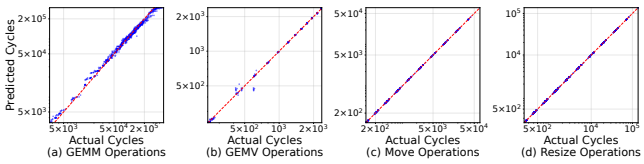


Figure 13. Performance model accuracy on WSE-3 for four kernel types across varying region sizes, comparing model predictions with real-hardware measurements. The model achieves 95–99% prediction accuracy, sufficient to guide schedule search.

Vector search. For 1M–20M vectors, Figure 12(c)(d) reports 6–59× lower simulated query latency than the same idealized reference and 21–204× higher estimated energy efficiency under the same ideal-scaling assumptions. The generated schedule assigns dense inner products and reductions to explicit mesh regions; the GPU-cluster comparison remains modeled rather than measured.

Takeaway. These case studies exercise MeshIR beyond transformer inference and show that the same CUT/TUNE representation can express irregular particle–mesh exchanges and dense reduction trees. They provide end-to-end workload-coverage evidence in the vendor SDK's cycle-level simulator; their GPU-cluster and energy comparisons remain subject to idealized scaling and power assumptions.

6.2 Search Accuracy and Efficiency

A practical wafer-scale compiler must produce good schedules and complete search in reasonable time. We therefore evaluate both cost-model accuracy and the convergence behavior of our hierarchical search procedure.

Performance Model Accuracy. We train a per-backend performance model to approximate the unimodal latency curve introduced in §4.3. Figure 13 compares predicted cycles with cycles measured on WSE-3 hardware for GEMM, GEMV, Move, and Resize across the region sizes and tensor shapes used in our evaluation; this comparison does not use simulator results. The points closely track the ideal diagonal. For components not included in the original regression figure, the model attains 99.0% prediction accuracy on elementwise and 96.4% on collective operations (0.99% and 3.60% mean absolute percentage error, respectively); separating Move by distance gives 96.8% accuracy for near transfers and 96.4% for long-range transfers. This accuracy is sufficient to rank candidate schedules.

Ablation Study. We ablate five schedule classes on one layer each from LLaMA3-8B and Qwen3-30B-A3B (MoE), covering decode at $K = 128/4096$ and prefill at $S = 128/2048$. Table 3 reports latency measured in the official Cerebras SDK simulator after schedule search under a fixed 500-iteration budget. **Full-Chip** uses the whole device serially; **Uniform** chooses one shared rectangle; and **Per-op** independently right-sizes operators while retaining serial execution. **+CUT** adds cross-operator decomposition with standalone TUNE disabled; **+TUNE** restores geometric refinement and yields full Wavel. Thus, the last two transitions isolate cross-operator scheduling and geometric refinement, respectively.

		Incremental latency reduction (%)				Overall
Model	**Workload**	**Uniform**	**Per-op**	**+CUT**	**+TUNE**	**speedup**
LLaMA3 8B	Decode, $K = 128$	18.3	7.0	24.7	9.7	1.94×
	Decode, $K = 4096$	17.4	4.9	23.8	5.8	1.77×
	Prefill, $S = 128$	18.3	6.0	25.8	8.3	1.92×
	Prefill, $S = 2048$	1.3	4.7	14.9	1.0	1.26×
Qwen3 30B-A3B	Decode, $K = 128$	55.4	0.3	9.9	24.5	3.30×
	Decode, $K = 4096$	50.1	1.9	8.4	18.9	2.75×
	Prefill, $S = 128$	49.9	3.1	19.8	20.3	3.22×
	Prefill, $S = 2048$	26.7	1.6	26.7	5.1	1.99×

Table 3. One-layer schedule ablation measured in the official Cerebras SDK simulator.

Optimization Stage	**8-Op MLP**	**LLaMA3-8B**
Baseline (brute force)	4.10×10^{51}	1.74×10^{2963}
+ Executable CUT/TUNE expansion	1.24×10^{17}	6.17×10^{763}
+ Unimodal geometric search	3.64×10^{14}	9.55×10^{621}
+ Structural hash and cache	8.80×10^{2}	4.23×10^{19}
+ Specialized subproblem optimization	3.53×10^{2}	3.68×10^{12}

Table 4. Analytical estimate of the candidate schedule space under successive structural restrictions; these are derived counts, not runtime search traces.

Table 3 shows workload-dependent contributions. For LLaMA3-8B decode and short prefill, CUT reduces latency measured in the SDK simulator by 23.8–25.8%, and full Wavel reaches 1.77–1.94× over Full-Chip. Longer prefill gains less from Uniform and TUNE (1.3% and 1.0%), but CUT still adds 14.9%. For Qwen3-30B-A3B, Uniform removes 49.9–55.4% and TUNE adds 20.3–24.5% at short settings; longer settings reach 2.75× in decode and 1.99× in prefill. The mechanisms are complementary, and one-layer speedups can exceed end-to-end gains because full-model execution includes schedule-insensitive stages.

Analytical Search-Space Reduction. Table 4 reports the analytical candidate-space reduction. The successive restrictions reduce the estimated candidate space by at least 49 orders of magnitude (about 49 for the 8-op MLP and 2,951 for LLaMA3-8B). These derived counts quantify the combinatorial space avoided, not states generated in a runtime trace.

6.3 Generalization Beyond Wafer Scale

We evaluate Wavel on the three Tenstorrent platforms described in the hardware setup above.

We compare against (i) **Tenstorrent Torch**, the standard PyTorch path that reloads from DRAM between operators, and (ii) **TTNN eager** (TT-Forge), which maximizes per-operator on-chip utilization but may evict tensors needed

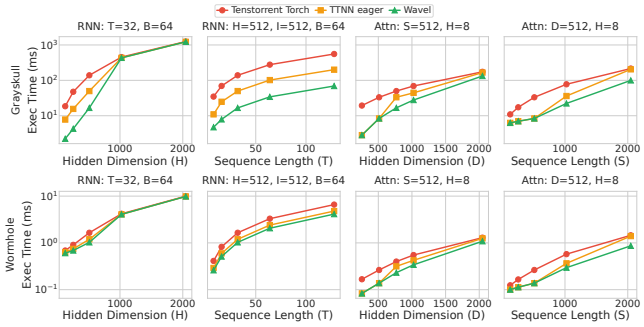


Figure 14. Execution time scaling on Tenstorrent Grayskull (top) and Wormhole (bottom). Wavel consistently outperforms both baselines, with the largest gains in high-reuse RNN scenarios where on-chip tensor retention eliminates over 99% of DRAM traffic.

Metric / Framework	LLaMA3.1-8B	Qwen3.5-4B
Device-side execution time (ms) per decode step		
TTNN	35.3 (0.8×)	181.7 (6.9×)
Torch-IR	67.8 (1.5×)	80.9 (3.1×)
Wavel	**46.2**	**26.4**
Host-side compilation time (s)		
vLLM (TT-Torch)	668.7 (27.6×)	682.0 (69.6×)
Wavel	**24.2**	**9.8**

Table 5. Tenstorrent Blackhole decode performance and compilation time. Wavel achieves up to 6.9× execution speedup and compiles 28–70× faster than the vendor pipeline, demonstrating that the MeshSchedule abstraction generalizes to smaller mesh accelerators.

later. As Figure 14 shows, the gains are largest in high-reuse RNN scenarios, where MeshSchedule-level co-scheduling keeps tensors on-chip across operators rather than evicting them to DRAM between dispatches.

Blackhole: End-to-End Decode and Compilation. We additionally evaluate on the newer **Tenstorrent Blackhole**, where Wavel compiles an entire operator DAG into a single dispatch. Table 5 compares device-side execution time per decode step against two baselines: (1) TTNN, the production vLLM [9] path using trace replay and hand-tuned fused operators, and (2) eager Torch-IR, a model-agnostic executor. We report device-side execution time per decode step and end-to-end host-side compilation time for both models. For LLaMA3.1-8B, TTNN trace replay achieves 35.3 ms, faster than Wavel's 46.2 ms, because it benefits from hand-tuned compound operators that reduce total dispatches. Against the model-agnostic Torch-IR path, Wavel achieves **1.5**× speedup. For Qwen3.5-4B, where no hand-tuned operator library exists, Wavel achieves **6.9**× and **3.1**× speedups over TTNN and Torch-IR respectively, demonstrating generalization to models lacking hand-optimized libraries.

The compilation-time advantage arises because Wavel emits a small number of generic kernel binaries and a compact schedule encoding, avoiding the per-operator compilation and dispatch setup that dominates the vLLM pipeline.

7 Related Work

Kernel scheduling systems. Prior AI compilers such as Halide [24], TVM [6], Ansor [33], Roller [37], TileLang [30], Triton [29], and XLA [21] focus primarily on optimizing individual operators or short fused regions. Their search spaces are highly effective for shared-memory GPUs/TPUs, but they do not provide an explicit device-wide schedule abstraction that jointly captures operator placement, spatial alignment, and SRAM resource conflicts across a large mesh.

Spatial DSLs and mesh programming systems. Prior spatial DSLs and compiler stacks such as AMOS [35], Groq [1], SambaNova [23], HeteroCL [10], and Dahlia [19] expose spatial structure more directly than GPU compilers do. However, these systems typically target smaller spatial fabrics or focus on kernel/library generation rather than a unified multi-kernel schedule abstraction for locality-sensitive mesh execution. Vendor wafer-scale stacks such as Cerebras Torch [4] only provide limited hard-coded models for LLM inference. Hand-optimized systems such as WaferLLM [7] demonstrate that careful mesh mapping matters, but they do not separate portable schedule semantics from backend-specific lowering in the way MeshIR does. More broadly, prior spatial systems address related concerns, but wafer scale is the regime where locality asymmetry, device-wide scale, and schedule-search complexity all become central at once.

Kernel mapping and compilation systems. Systems such as T10 [16] move beyond per-operator optimization and reason about dataflow across distributed on-chip resources. Wavel shares this device-wide perspective, but focuses on wafer-scale mapping, where hop-distance-sensitive locality, explicit alignment of producer/consumer layouts, and resource-dependence edges must all be represented inside the same schedule object. This is the main abstraction difference between Wavel and prior device-mapping systems.

Distributed sharding and planning. Systems such as Mesh-TensorFlow, GShard [13], Alpa [34], and nnScaler [14] automate tensor parallelism across multi-device clusters. These frameworks are designed around inter-device sharding and relatively coarse communication tiers. They do not model the fine-grained, hop-dependent behavior of on-chip 2D meshes, nor do they expose region-level resource conflicts or explicit spatial-alignment kernels as part of the executable schedule. The latency and bandwidth characteristics of on-chip meshes (cycle-level, locality-sensitive) differ fundamentally from those of datacenter networks (microsecond–millisecond, tiered), placing Wavel in a distinct compilation regime.

8 Discussion and Future Work

Abstraction boundary and portability. The central claim of Wavel is that MeshSchedule provides a stable design boundary: locality, alignment, and resource conflicts are explicit, while CUT and TUNE define legal refinements over that object. Operator modeling and movement calibration change across targets. The current implementation provides code-generation paths for both backends: statically routed CSL programs on Cerebras, and reader, writer, and compute kernels with runtime dispatch on Tenstorrent. This demonstrates that the shared search abstraction ports across qualitatively different execution models.

Limitations. Several of Wavel's restrictions are deliberate, and they bound what our results establish.

Restricted schedule space and no optimality guarantee. CUT only divides a rectangular allocation into two adjacent rectangles along a full horizontal or vertical cut, and TUNE resizes a region around a fixed corner. Layouts that require disconnected or more irregular pieces, or moving that corner, are therefore not explored. Search within this space is also heuristic: unimodality (§4.3.2) is empirical rather than proven, so ternary search may settle in a local optimum; bounded A* stops at a fixed iteration budget; and directly fixing low-complexity subproblems (§5) commits to one solution. Table 4 counts the space these restrictions remove, not the quality of what remains; we claim no global optimality.

MoE code-generation maturity. MeshIR schedules the main expert computation, but end-to-end MoE execution also contains router operators, top-k selection, and dynamic token dispatch. These steps introduce code-generation edge cases that sometimes require manual tuning of the generated code. Recent iterations have resolved most of them, but robust large-batch MoE inference still needs additional runtime and engineering support. We therefore present Qwen3-30B-A3B and GPT-OSS-20B as automatic schedule-generation and calibrated cost-model results (Figure 10), not as fully automatic, measured end-to-end code generation.

Limits of automatic kernel fusion. Wavel sometimes cannot match the degree of operator fusion achieved by vendor hand-tuned kernels. For example, on Tenstorrent Blackhole, TTNN's hand-tuned trace-replay path uses fused compound operators to reduce dispatch overhead beyond what our current backend generates automatically, and therefore outperforms Wavel on LLaMA3.1-8B (Table 5). This result shows that mature, model-specific vendor kernels can outperform Wavel when their hand-engineered kernel fusion is more aggressive than the fusion available in our current backend.

Fault-map awareness. The current implementation does not optimize placement or routing against a device fault map; fault-map-aware placement and routing are left to future work.

Extending to higher-dimensional mesh architectures. Although Wavel focuses on 2D meshes, MeshIR and the CUT/TUNE framework generalize naturally to meshes with more axes. A Region extends from a 2D rectangular block to a contiguous block along any number of mesh axes; hop-aware communication extends by counting hops per axis. Crucially, the schedule description grows with the number of mesh axes, not raw mesh size: the number of coordinates per Region scales with the number of axes, preserving the same CUT/TUNE transition structure and decomposition-based pruning mechanisms. This structural property gives Wavel a practical portability path as mesh accelerator architectures evolve beyond 2D.

Adaptive scheduling under workload variation. A consequence of explicit region placement is that optimal region sizes and spatial alignments are sensitive to workload characteristics. Changes in sequence length, batch size, or the number of activated experts in MoE models can shift the bottleneck between compute and communication, altering ideal region shapes, placements, and spatial alignment kernels. This makes static schedules suboptimal under workload variation, for example across fluctuating batch sizes [32] or mixed prefill and decode phases [15]. The current implementation generates schedules offline for known shapes; online rescheduling and schedule bucketing are future work. Wavel's cost model and CUT/TUNE transition rules provide a natural basis for adaptive scheduling: the compiler could maintain a small set of pre-compiled MeshSchedules and switch among them as workload characteristics change, or refine region geometries at runtime using the same physical cost evaluation developed in this work.

9 Conclusion

Taken together, our results suggest that wafer-scale compilation requires a new foundation beyond conventional sharding-oriented designs. By making executable, locality-preserving schedules the central compiler object, Wavel shows that high-quality wafer-scale mappings can be found efficiently across a range of LLMs and wafer-scale workloads, matching or surpassing solutions that previously required weeks of human expert effort or search with alternative methods. We view this as an important step toward practical software support for this emerging class of hardware.

Looking forward, Wavel also opens up a broader agenda for locality-sensitive wafer-scale compilation and for emerging mesh-based architectures that are likely to play an increasingly important role in future accelerator designs. We hope its abstractions provide a foundation for future work on AI-driven compilation and emerging larger-scale accelerators with non-uniform memory architectures.

References

[1] Dennis Abts, Garrin Kimmell, Andrew Ling, John Kim, Matt Boyd, Andrew Bitar, Sahil Parmar, Ibrahim Ahmed, Roberto DiCecco, David Han, et al. 2022. A software-defined tensor streaming multiprocessor

for large-scale machine learning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 567–580.

[2] Advanced Micro Devices. 2023. *AMD Optimizes EPYC Memory with NUMA*. White Paper. Advanced Micro Devices, Inc. https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/AMD-Optimizes-EPYC-Memory-With-NUMA.pdf

[3] Amazon Web Services. 2026. AWS and Cerebras Collaboration Aims to Set a New Standard for AI Inference Speed and Performance in the Cloud. https://press.aboutamazon.com/aws/2026/3/aws-and-cerebras-collaboration-aims-to-set-a-new-standard-for-ai-inference-speed-and-performance-in-the-cloud. Accessed: 2026-04-02.

[4] Cerebras Systems. 2020. Supporting PyTorch on the Cerebras Wafer Scale Engine. https://www.cerebras.ai/blog/supporting-pytorch-on-the-cerebras-wafer-scale-engine. Accessed: 2025-12-11.

[5] Cerebras Systems. 2025. Cerebras Brings Instant Inference to Mistral Le Chat. https://www.cerebras.ai/blog/mistral-le-chat. Accessed: 2026-04-02.

[6] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, 578–594. https://www.usenix.org/conference/osdi18/presentation/chen

[7] Congjie He, Yeqi Huang, Pei Mu, Ziming Miao, Jilong Xue, Lingxiao Ma, Fan Yang, and Luo Mai. 2025. WaferLLM: Large Language Model Inference at Wafer Scale. *arXiv preprint arXiv:2502.04563* (2025).

[8] Hyoukjun Kwon, Prasanth Chatarasi, Vivek Sarkar, Tushar Krishna, Michael Pellauer, and Angshuman Parashar. 2020. Maestro: A data-centric approach to understand reuse, performance, and hardware cost of DNN mappings. *IEEE Micro* 40, 3 (2020), 20–29.

[9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

[10] Yi-Hsiang Lai, Yuze Chi, Yuwei Hu, Jie Wang, Cody Hao Yu, Yuan Zhou, Jason Cong, and Zhiru Zhang. 2019. HeteroCL: A multi-paradigm programming infrastructure for software-defined reconfigurable computing. In *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 242–251.

[11] Mark LaPedus. 2023. TSMC bets big on advanced packaging. https://spectrum.ieee.org/tsmc-advanced-packaging. Accessed: 2024-11-29.

[12] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2020. MLIR: A compiler infrastructure for the end of Moore's law. *arXiv preprint arXiv:2002.11054* (2020).

[13] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. GShard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).

[14] Zhiqi Lin, Youshan Miao, Quanlu Zhang, Fan Yang, Yi Zhu, Cheng Li, Saeed Maleki, Xu Cao, Ning Shang, Yilei Yang, et al. 2024. nnScaler: Constraint-Guided Parallelization Plan Generation for Deep Learning Training. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 347–363.

[15] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[16] Yiqi Liu, Yuqi Xue, Yu Cheng, Lingxiao Ma, Ziming Miao, Jilong Xue, and Jian Huang. 2024. Scaling deep learning computation over the inter-core connected intelligence processor with T10. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 505–521.

[17] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. 2020. Rammer: Enabling Holistic Deep Learning Compiler Optimizations with rTasks. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 881–897.

[18] Meta and Cerebras Systems. 2025. Meta Collaborates with Cerebras to Drive Fast Inference for Developers in New Llama API. https://www.businesswire.com/news/home/20250429462666/en/Meta-Collaborates-with-Cerebras-to-Drive-Fast-Inference-for-Developers-in-New-Llama-API. Accessed: 2026-04-02.

[19] Rachit Nigam, Sachille Atapattu, Samuel Thomas, Zhijing Li, Theodore Bauer, Yuwei Ye, Apurva Koti, Adrian Sampson, and Zhiru Zhang. 2020. Predictable accelerator design with time-sensitive affine types. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. 393–407.

[20] OpenAI. 2026. OpenAI Partners with Cerebras. https://openai.com/index/cerebras-partnership/. Accessed: 2026-04-02.

[21] OpenXLA Project. 2024. XLA: Accelerated Linear Algebra. https://openxla.org/xla.

[22] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W Keckler, and Joel Emer. 2019. Timeloop: A systematic approach to DNN accelerator evaluation. In *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 304–315.

[23] Raghu Prabhakar, Ram Sivaramakrishnan, Darshan Gandhi, Yun Du, Mingran Wang, Xiangyu Song, Kejie Zhang, Tianren Gao, Angela Wang, Xiaoyan Li, et al. 2024. SambaNova SN40L: Scaling the AI memory wall with dataflow and composition of experts. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1353–1366.

[24] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *ACM SIGPLAN Notices* 48, 6 (2013), 519–530.

[25] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. 2023. Welder: Scheduling Deep Learning Memory Access via Tile-graph. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 701–718.

[26] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[27] Emil Talpes, Douglas Williams, and Debjit Das Sarma. 2022. DOJO: The Microarchitecture of Tesla's Exa-Scale Computer. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. 1–28.

[28] Tenstorrent. 2025. tt-torch Documentation. https://docs.tenstorrent.com/tt-torch/. Accessed: 2025-12-11.

[29] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. 10–19.

[30] Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, et al. 2025. TileLang: A Composable Tiled Programming Model for AI Systems. *arXiv preprint arXiv:2504.17577* (2025).

[31] Lei Wang, Lingxiao Ma, Shijie Cao, Quanlu Zhang, Jilong Xue, Yining Shi, Ningxin Zheng, Ziming Miao, Fan Yang, Ting Cao, et al. 2024. Ladder: Enabling Efficient Low-Precision Deep Learning Computing through Hardware-aware Tensor Transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI*

24). 307–323.

[32] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX symposium on operating systems design and implementation (OSDI 22)*. 521–538.

[33] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 863–879. https://www.usenix.org/conference/osdi20/presentation/zheng

[34] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating inter- and Intra-Operator parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.

[35] Size Zheng, Renze Chen, Anjiang Wei, Yicheng Jin, Qin Han, Liqiang Lu, Bingyang Wu, Xiuhong Li, Shengen Yan, and Yun Liang. 2022. AMOS: enabling automatic mapping for tensor computations on spatial accelerators with hardware abstraction. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 874–887.

[36] Size Zheng, Yun Liang, Shuo Wang, Renze Chen, and Kaiwen Sheng. 2020. FlexTensor: An Automatic Schedule Exploration and Optimization Framework for Tensor Computation on Heterogeneous System. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 859–873.

[37] Hongyu Zhu, Ruofan Wu, Yijia Diao, Shanbin Ke, Haoyu Li, Chen Zhang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Wei Cui, Fan Yang, Mao Yang, Lidong Zhou, Asaf Cidon, and Gennady Pekhimenko. 2022. ROLLER: Fast and Efficient Tensor Compilation for Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 233–248.